Physics Informed Neural Network (PINN) - Introduction

Paweł Maczuga[1]

[1]Institute of Computer Science
AGH University of Kraków, Poland

December 3, 2025

Table of Contents

Introduction

What are Physics-Informed Neural Networks?
or **PINNs** in short

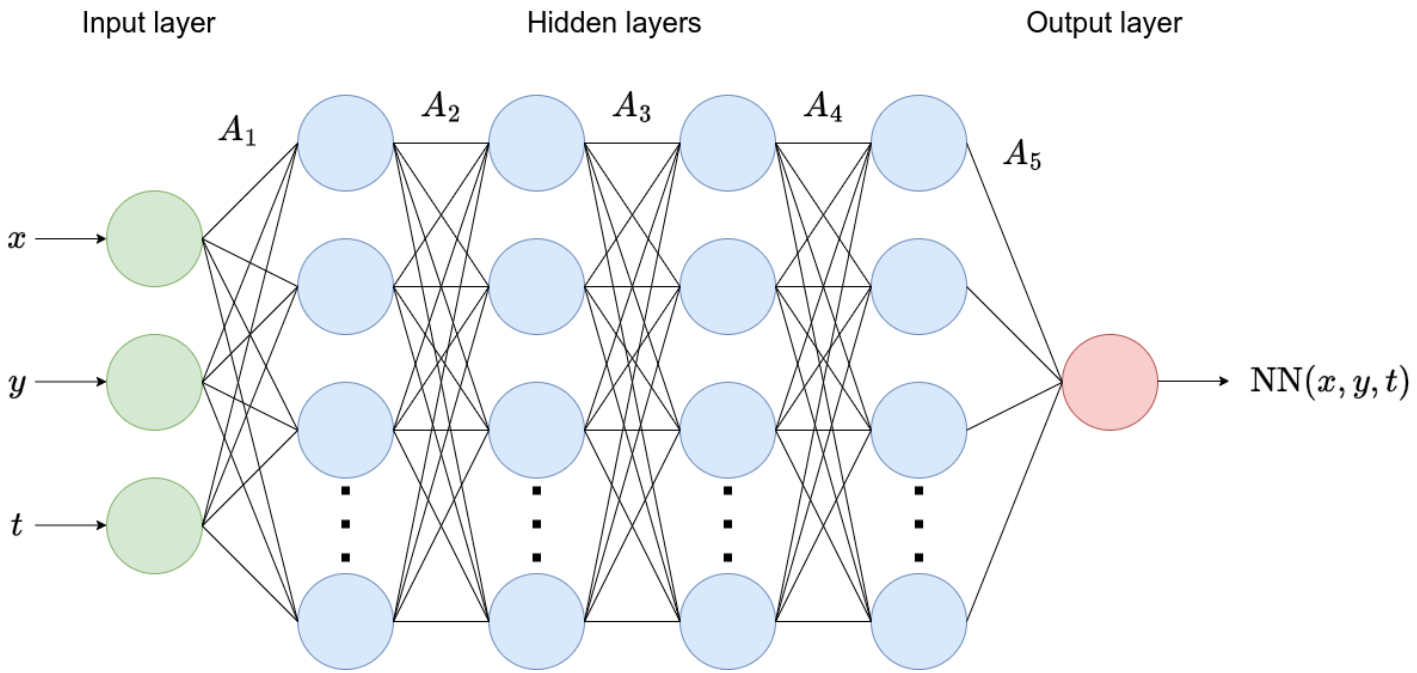
- Neural Networks that learn the solution to PDE (Partial Differential Equation)
- Without the need for any data.
- Can be used to perform simulations.

Table of Contents

Neural Network

NN is a nonlinear **function** - transforms input to output.
So, it can approximate a solution to PDE.

$$\mathrm{NN}(x, y, t) = A_n \ldots \sigma(A_3\sigma(A_2\sigma(A_1 \begin{bmatrix} x \\ y \\ t \end{bmatrix} + b_1) + b_2) + b_3) + \cdots + b_n$$



Neural Network training

- Choose collocation points $c_p = \{(x_1, y_1, t_1), (x_2, y_2, t_2), \cdots (x_n, y_n, t_n)\}$
- Compute NN solution $NN(x, y, t) \quad \forall (x, y, t) \in c_p$
- Compute LOSS function in c_p - how well the NN performs
- Update NN weights - matrices A_i, vectors b_i
  - Based on derivatives of LOSS with respect to weights
  - multiplied by learning rate (training parameter)

What is Physics Informed Neural Network

Physics Informed Neural Network (PINN in short)

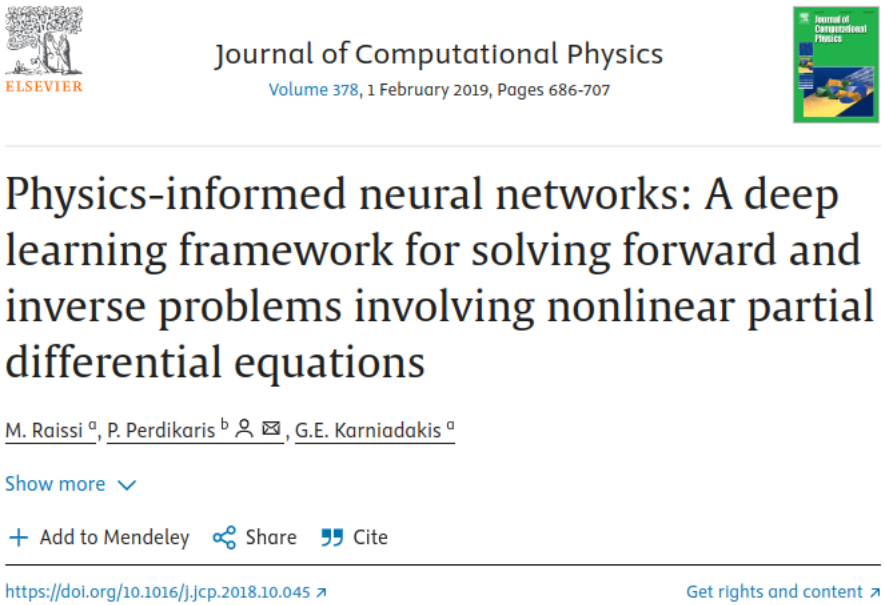
- It is a neural network that learns the solution to a Partial Differential Equation (PDE)
- Under the hood, it is simply a Feed Forward NN
- Physics is "injected" in the loss function
- PINN can learn the PDE **without** forming and solving system of linear equations

Putting it together:
NN represents the solution scalar or vector field in PDE and it learns the solution based on the physics described by PDE.

Physics Informed Neural Networks - PINN

It was introduced by George Karniadakis in 2019:

M. Raissi, P. Perdikaris, G.E.Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs, **Journal of Computational Physics** 378(1) (2019)



Loss function

The loss function depends on specific PDE and contains 3 parts:

- PDE loss - $LOSS_{PDE}$
- Initial condition loss - $LOSS_{IC}$
- Boundary condition loss - $LOSS_{BC}$

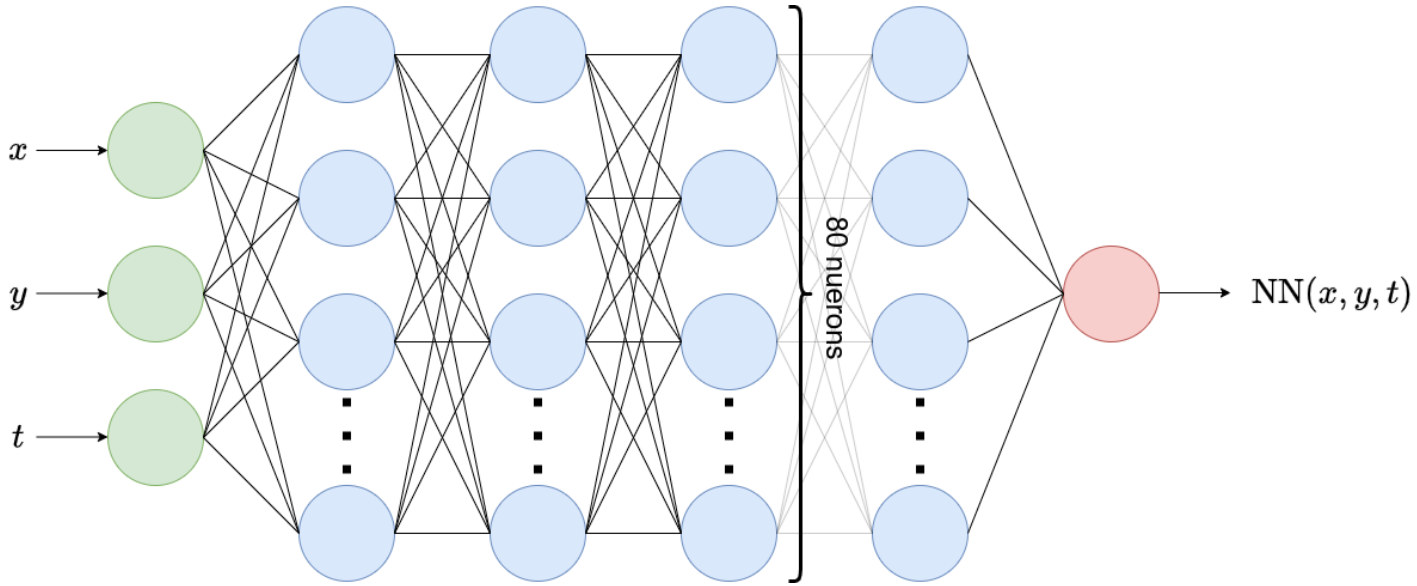
$$LOSS = LOSS_{PDE} + LOSS_{IC} + LOSS_{BC}$$

We train the NN to minimize this loss function.

Table of Contents

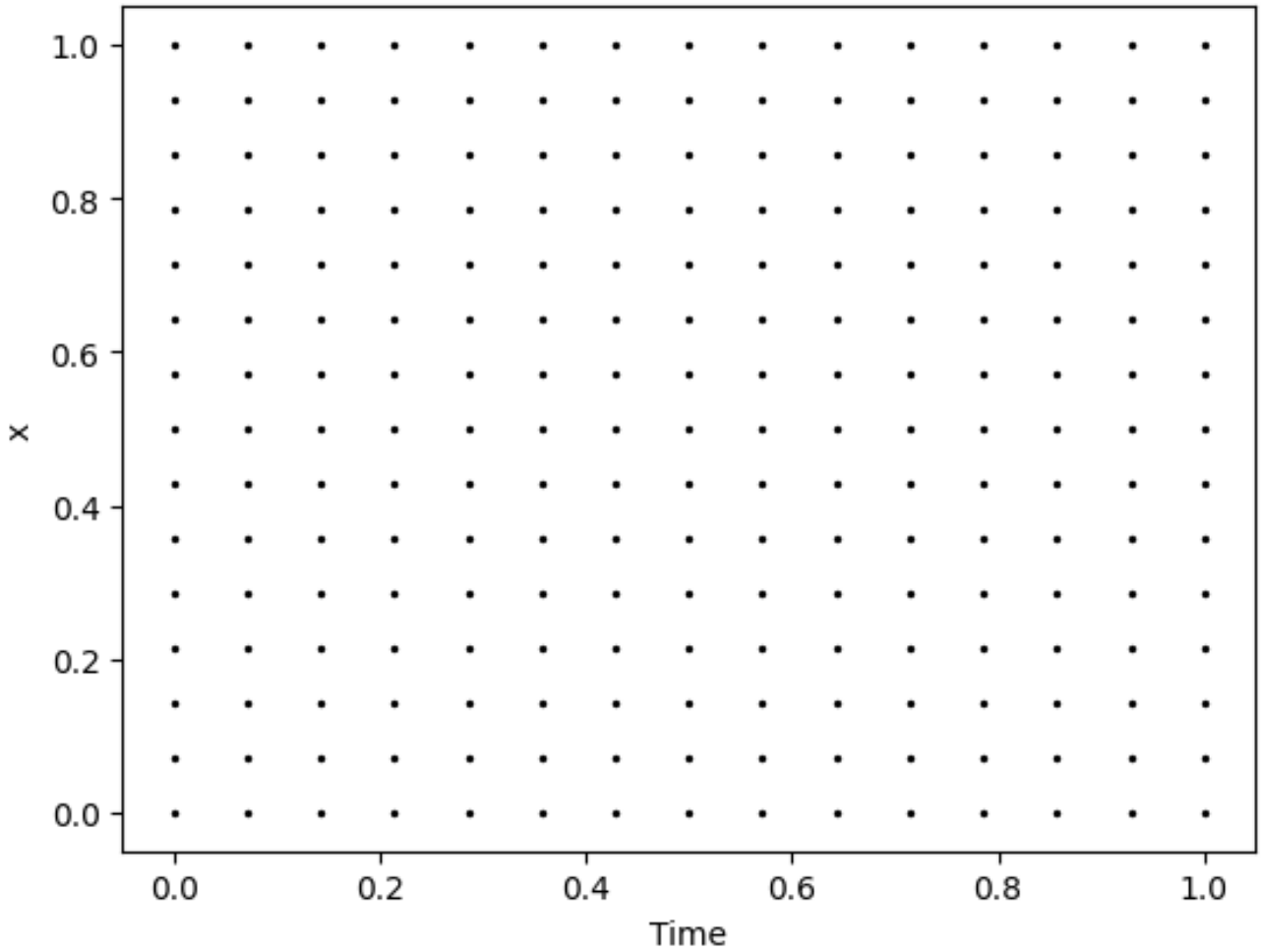
Network architecture

- Number of hidden layers: 4
- Neurons per layer: 80
- Activation function: **tanh**[1]



[1] Maciej Paszyński Paweł Maczuga. "C Influence of Activation Functions on the Convergence of Physics-Informed Neural Networks for 1D Wave Equation". In: *Computational Science – ICCS 2023: 23rd International Conference, Prague, Czech Republic, July 3-5, 2023, Proceedings, Part I* (2023), pp. 74–88.

Collocation points



PDE loss function

Equation:

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0, \qquad [0,1] \times [0,1]$$

Residual squared - PDE loss:

$$LOSS_{PDE}(x,t) = \left(\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} \right)^2$$

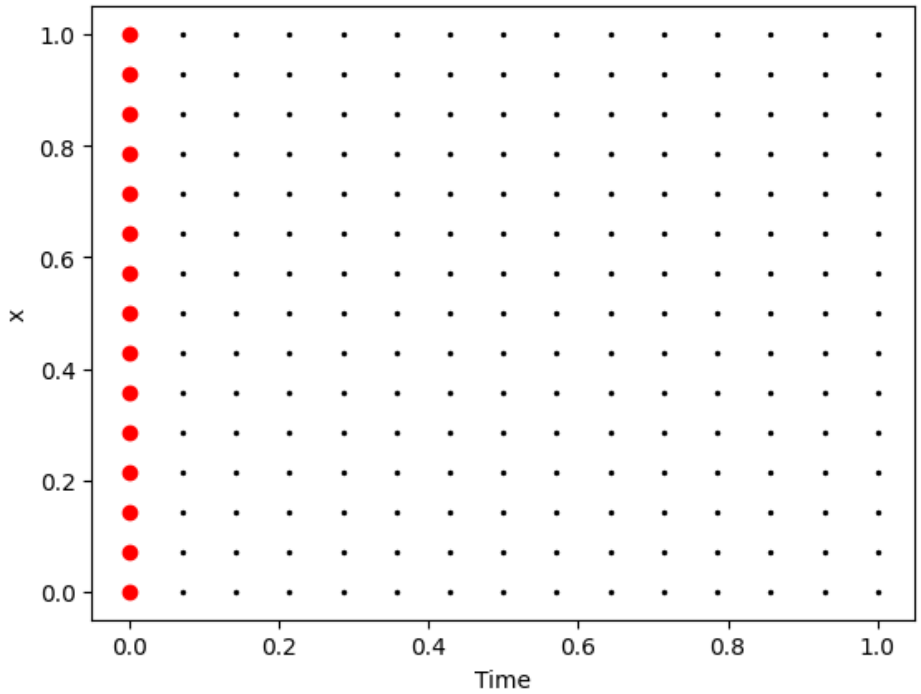
Initial condition loss function

Initial condition:

$$u(x,0) = u_0(x), \qquad x \in [0,1].$$

Initial condition loss:

$$LOSS_{Init}(x,0) = (PINN(x,0) - u_0(x))^2$$



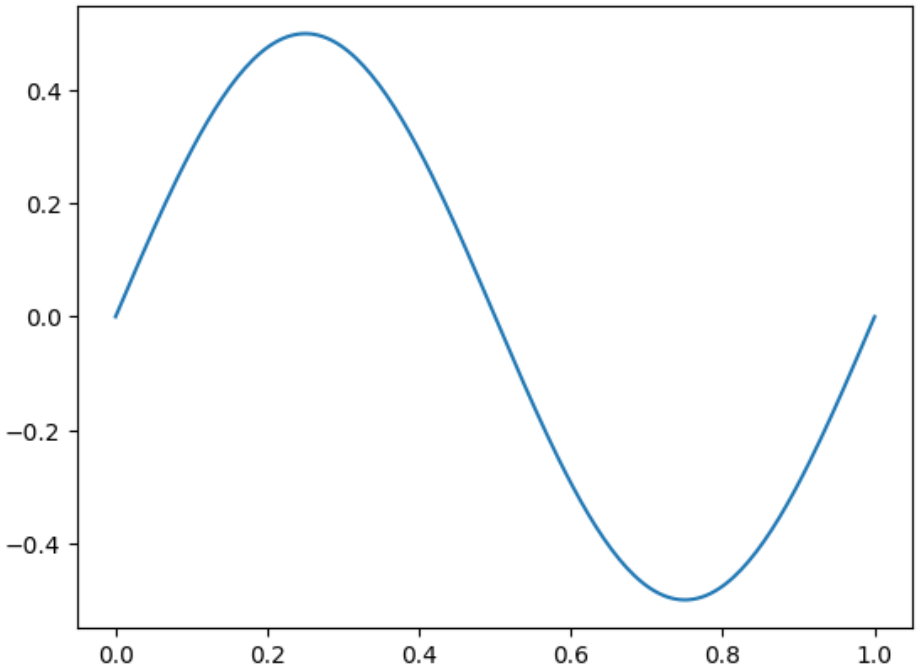
Initial condition

Initial condition:

$$u0(x) = 0.5 \sin(2\pi x)$$

Initial condition loss:

$$LOSS_{Init}(x, 0) = (PINN(x, 0) - 0.5 \sin(2\pi x))^2$$



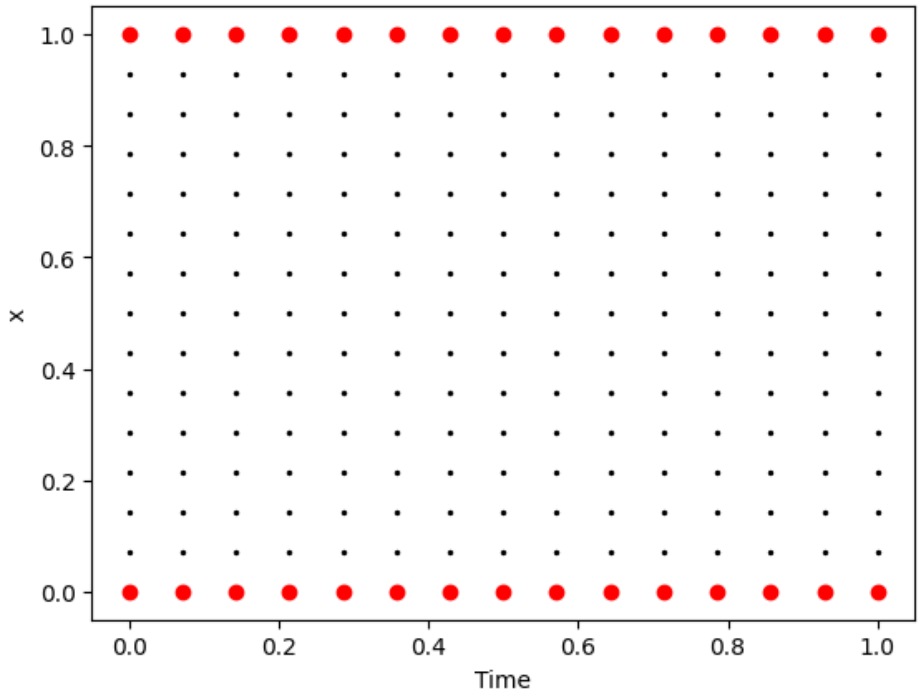
Boundary condition loss function

Boundary condition:

$$u(x,t) = 0, \qquad \text{on } \partial\Omega \times [0,1].$$

Boundary condition loss:

$$LOSS_{BC}(x,t) = \left(PINN(x,t) - 0\right)^2.$$



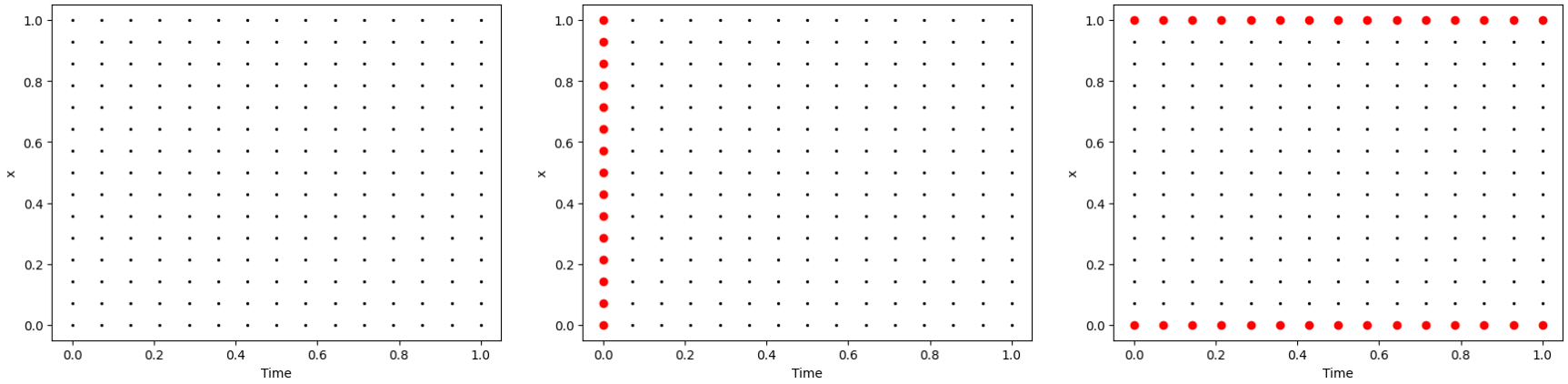
Total loss function

Total loss function:

$$LOSS = LOSS_{PDE} + LOSS_{IC} + LOSS_{BC}$$

To improve the convergence, we can add weights:

$$LOSS = w_{PDE} LOSS_{PDE} + w_{IC} LOSS_{IC} + w_{BC} LOSS_{BC}$$



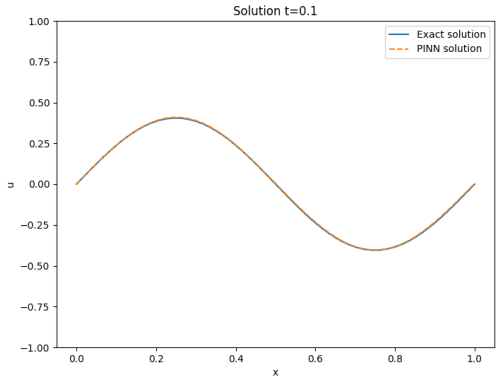
Additional parameters

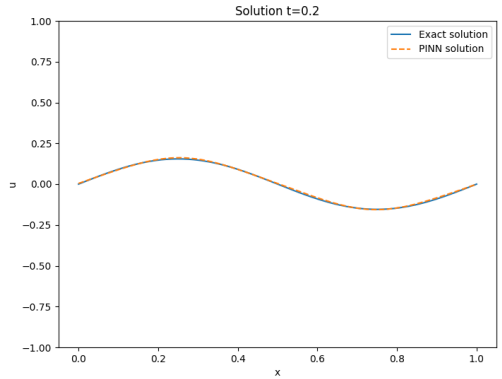
Parameters:

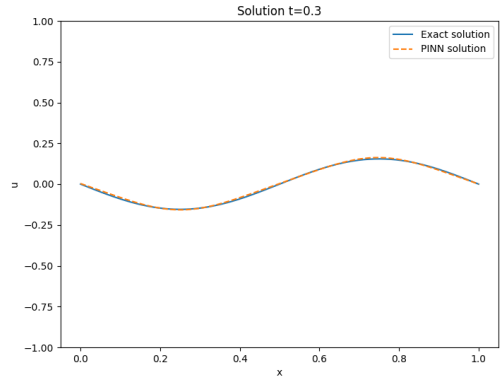
- ADAM optimizer
- Maximum epochs: 20 000,
- Learning rate: $\eta = 0.002$
- Weights $= 1$
- Number of hidden layers: 4
- Neurons per layer: 80
- Activation function: tanh

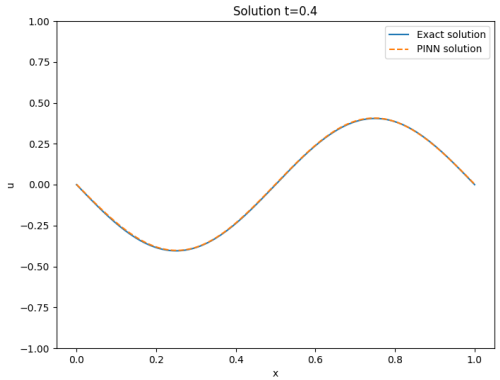
Table of Contents

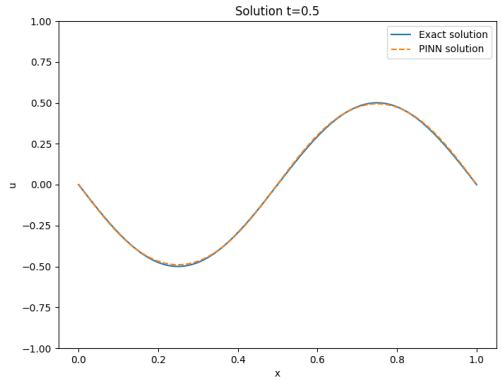
Results

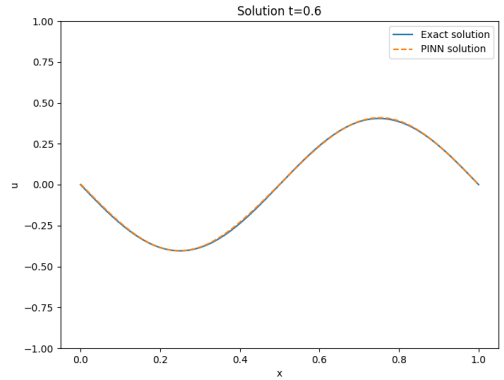

Solution t=0.1
Exact solution
PINN solution
u
x

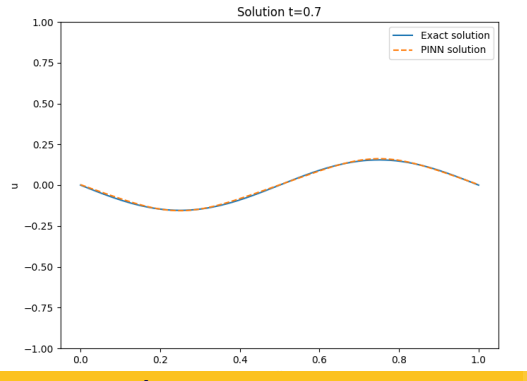

Solution t=0.2
Exact solution
PINN solution
u
x

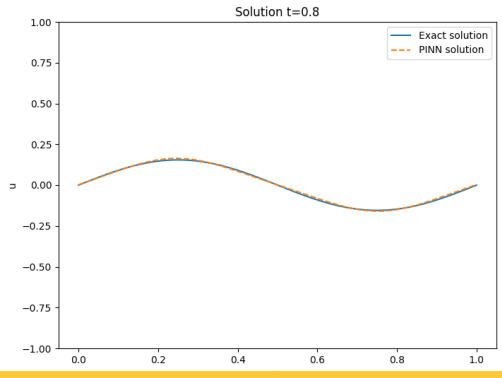

Solution t=0.3
Exact solution
PINN solution
u
x

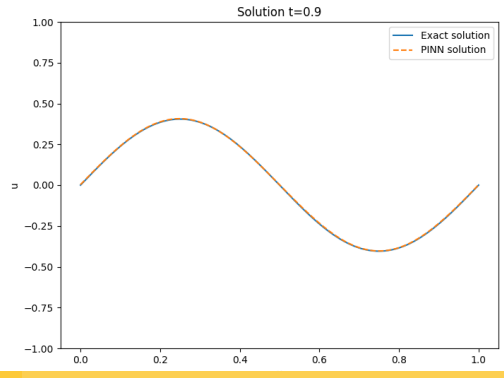

Solution t=0.4
Exact solution
PINN solution
u
x


Solution t=0.5
Exact solution
PINN solution
u
x

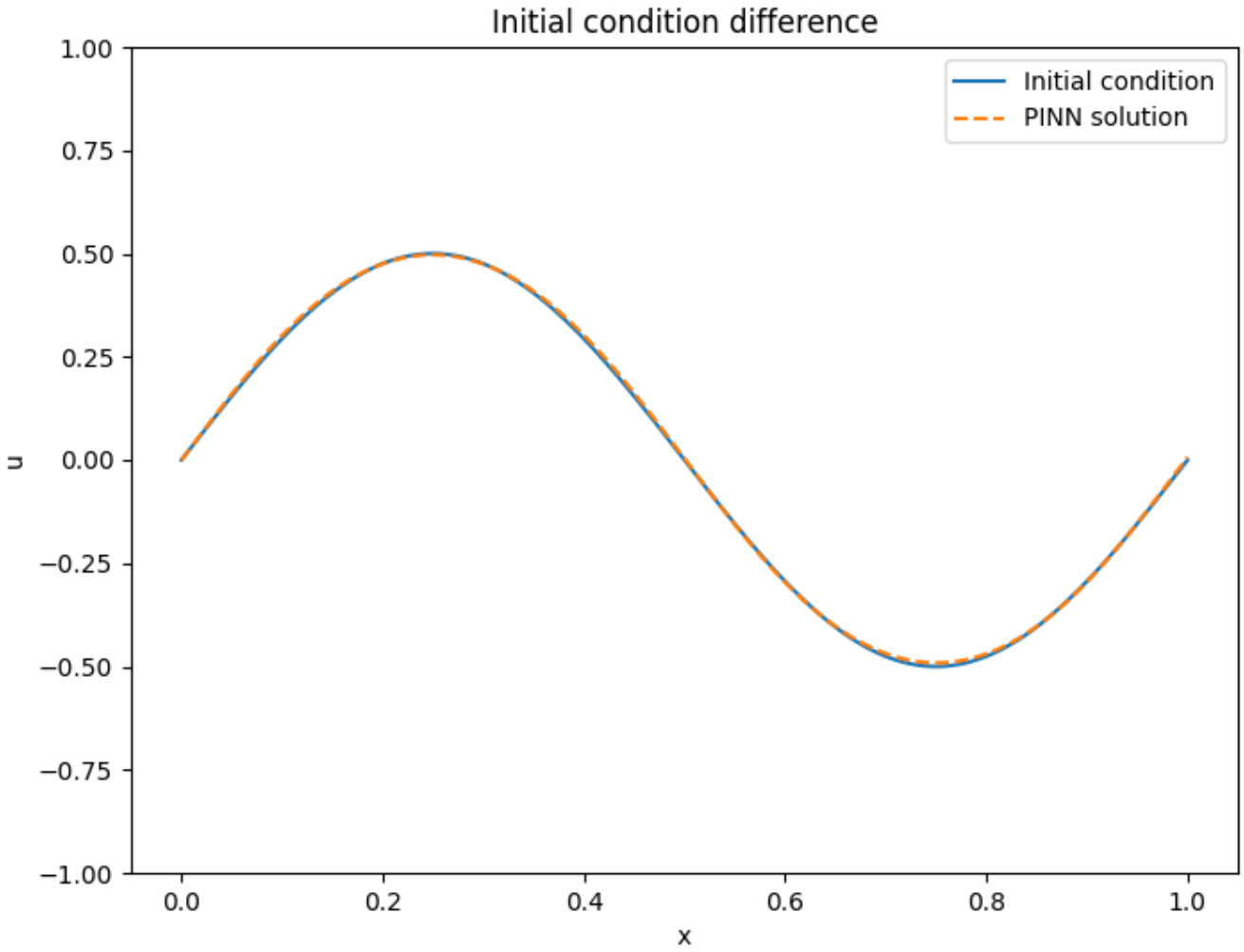

Solution t=0.6
Exact solution
PINN solution
u
x


Solution t=0.7
Exact solution
PINN solution
u

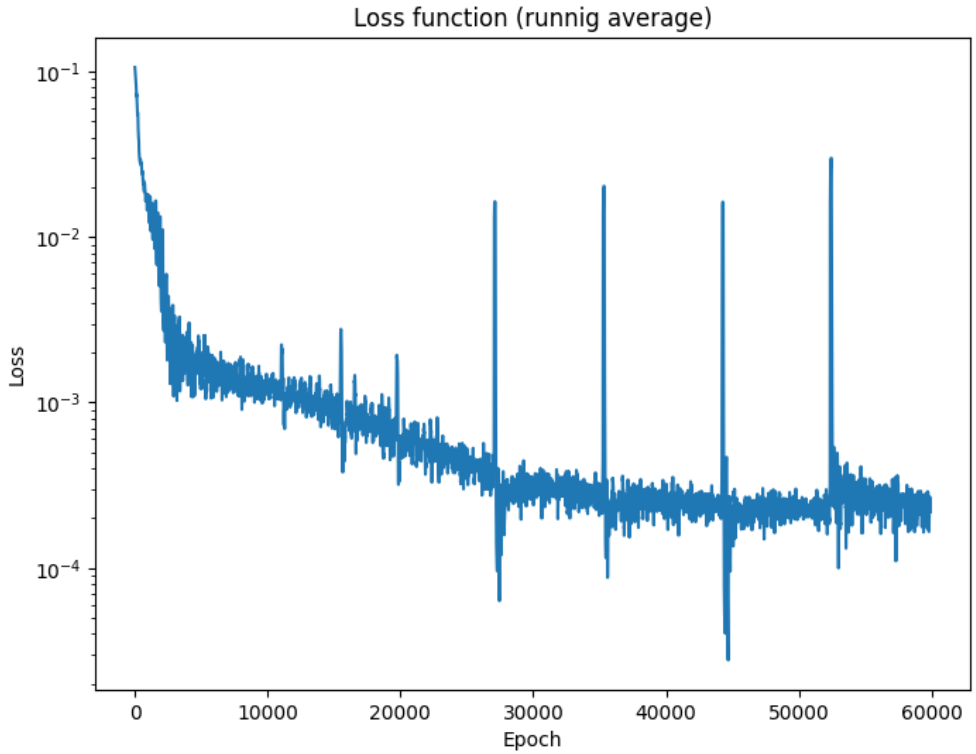

Solution t=0.8
Exact solution
PINN solution
u


Solution t=0.9
Exact solution
PINN solution
u

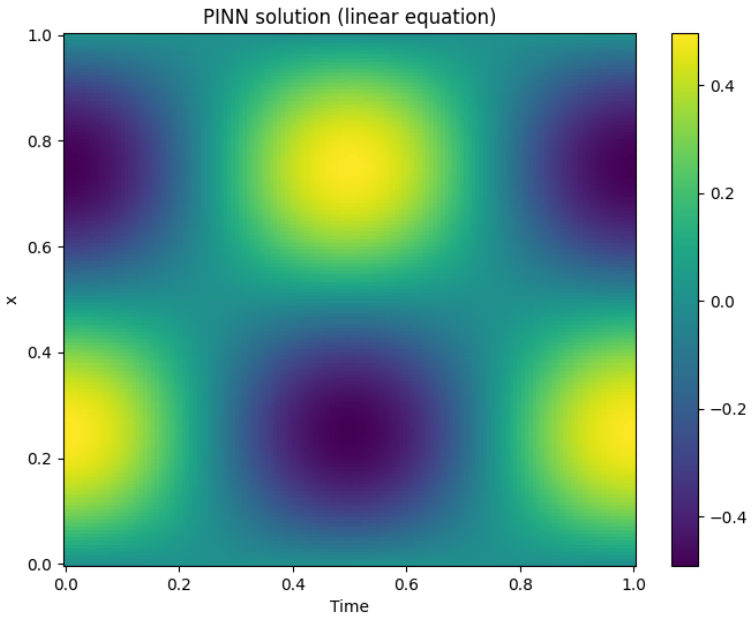
Initial condition

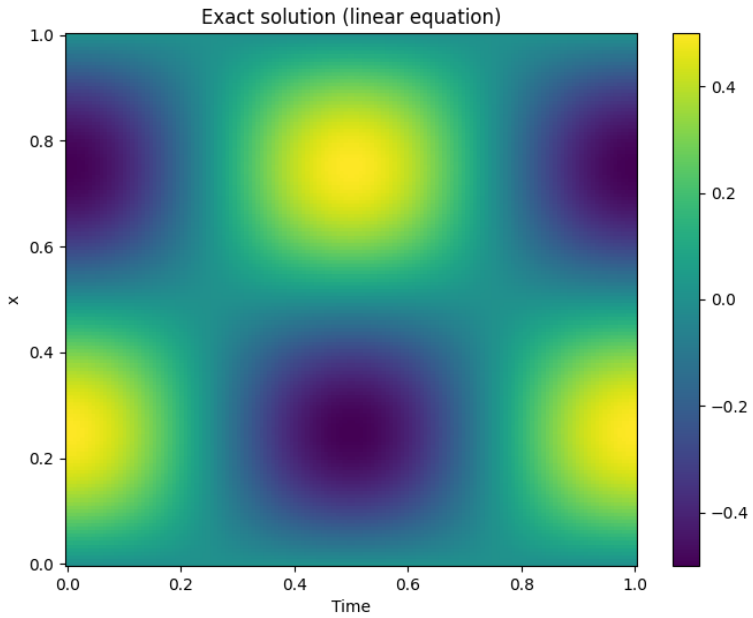


Loss


Loss function (runnig average)
Loss
10^{-1}
10^{-2}
10^{-3}
10^{-4}
0
10000
20000
30000
40000
50000
60000
Epoch

Solutions in 2D





Difference between PINN and exact

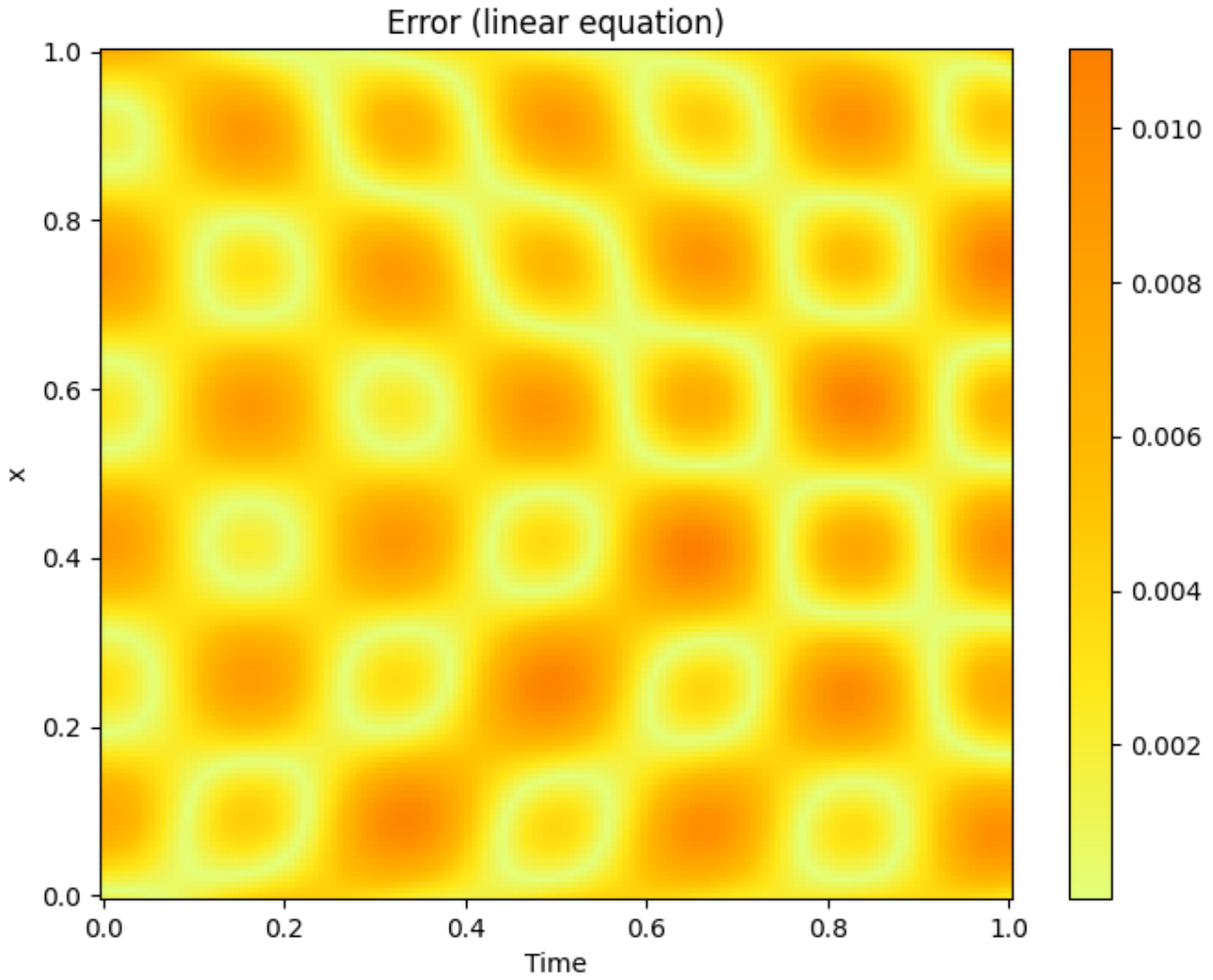


Table of Contents

Code structure

Code is a single Jupyter file divided into the following sections

- Parameters.
- Implementation.
  - Initial condition.
  - PINN
  - Loss function.
  - Train function
  - Plotting functions.
- Running code.
- Plotting.

Parameters

```
# Parameters
LENGTH = 2                  # Domain size (x axis). Starts at 0
TOTAL_TIME = .5             # Domain size (t axis). Starts at 0
N_POINTS = 15               # Number of in single asxis
N_POINTS_PLOT = 150         # Points in plotting(single axis)
WEIGHT_RESIDUAL = 0.03      # Weight of residual part of loss
WEIGHT_INITIAL = 1.0        # Weight of initial part of loss
WEIGHT_BOUNDARY = 0.005     # Weight of boundary part of loss
LAYERS = 10
NEURONS_PER_LAYER = 120
EPOCHS = 150_000
LEARNING_RATE = 0.00015
GRAVITY=9.81
```

PINN class

```
class PINN(nn.Module):
    def __init__(self, num_hidden, dim_hidden, act=nn.Tanh
        ()):
        ...

    def forward(self, x, y, t):
        x_stack = torch.cat([x, y, t], dim=1)
        out = self.act(self.layer_in(x_stack))
        for layer in self.middle_layers:
            out = self.act(layer(out))
        logits = self.layer_out(out)

        return logits
```

Simply Feed-Forward NN.

Derivation

```
def df(output, input, order = 1):
    ...
    torch.autograd.grad(...)
    ...

def dfdt(pinn, x, y, t, order = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, t, order=order)

def dfdx(pinn, x, y, t, order = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, x, order=order)

def dfdy(pinn, x, y, t, order = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, y, order=order)
```

PDE Loss

```
class Loss:
    ...
    def residual_loss(self, pinn: PINN):
        x, y, t = get_interior_points(self.x_domain, self.
            y_domain, self.t_domain, self.n_points, pinn.
            device())
        loss = dfdt(pinn, x, y, t) - dfdx(pinn, x, y, t,
            order=2) - dfdy(pinn, x, y, t, order=2)
        return loss.pow(2).mean()

    ...

    def __call__(self, pinn: PINN):
        ...
```

Boundary Loss

```
class Loss:
    ...

    def boundary_loss(self, pinn: PINN):
        ...

        loss_down  = dfdy(pinn, x_down,  y_down,  t_down )
        loss_up    = dfdy(pinn, x_up,    y_up,    t_up   )
        loss_left  = dfdx(pinn, x_left,  y_left,  t_left )
        loss_right = dfdx(pinn, x_right, y_right, t_right)

        return loss_down.pow(2).mean()  + \
            loss_up.pow(2).mean()     + \
            loss_left.pow(2).mean()   + \
            loss_right.pow(2).mean()

    ...
```

Boundary Loss

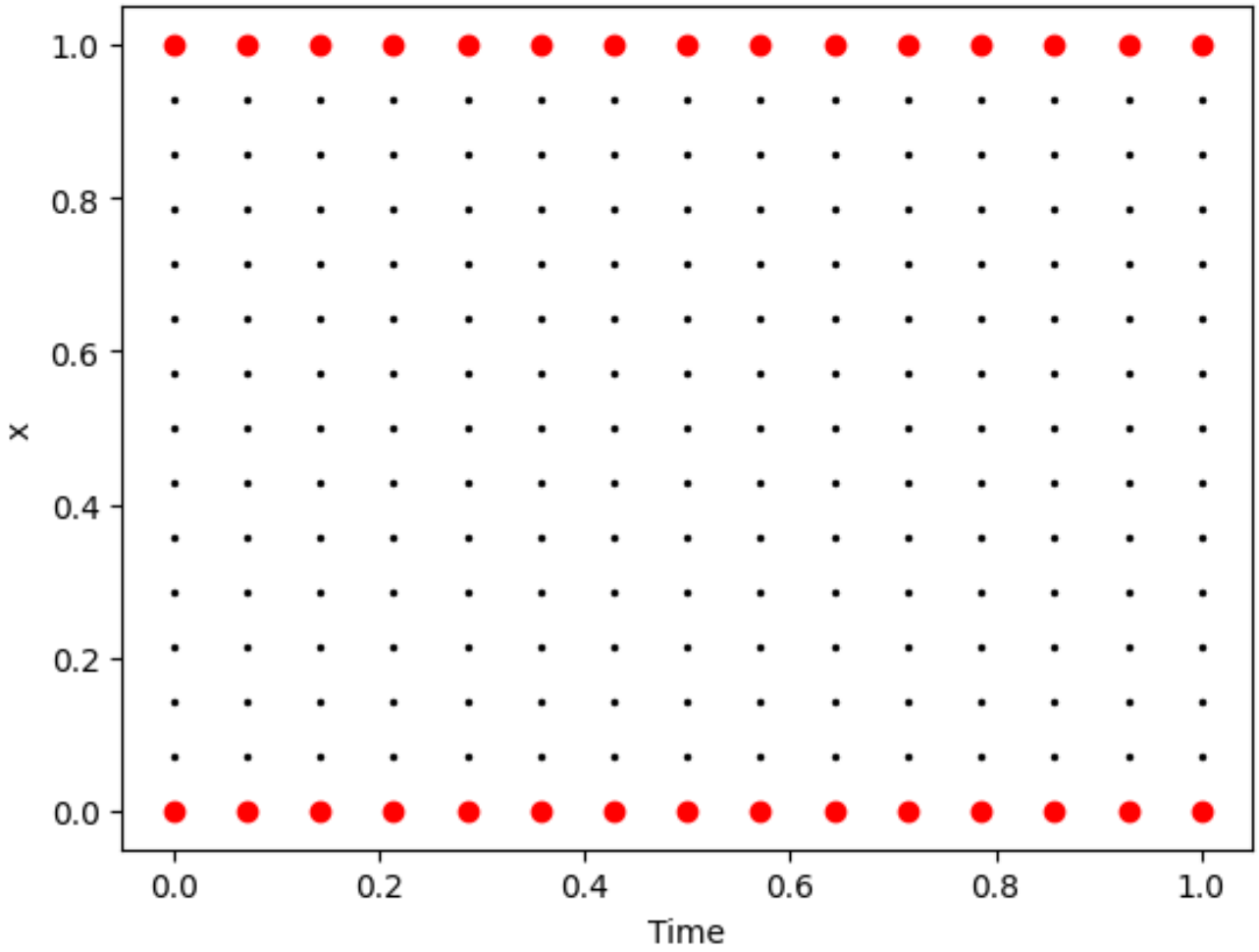


Figure 2: Collocation points on boundary.

Initial Loss

```
class Loss:
    ...

    def initial_loss(self, pinn: PINN):
        x, y, t = get_initial_points(self.x_domain, self.
            y_domain, self.t_domain, self.n_points, pinn.
            device())
        pinn_init = self.initial_condition(x, y)
        loss = f(pinn, x, y, t) - pinn_init
        return loss.pow(2).mean()

    ...
```

Training

```
pinn = PINN(LAYERS, NEURONS_PER_LAYER, act=nn.Tanh()).to(
    device)

x_domain = [0.0, LENGTH]
y_domain = [0.0, LENGTH]
t_domain = [0.0, TOTAL_TIME]

# train the PINN
loss_fn = Loss(x_domain, y_domain, t_domain,
    N_POINTS, initial_condition,
    WEIGHT_RESIDUAL, WEIGHT_INITIAL, WEIGHT_BOUNDARY
)

pinn_trained, loss_values = train_model(
    pinn, loss_fn=loss_fn, learning_rate=LEARNING_RATE,
        max_epochs=EPOCHS)
```

Training

```
Epoch: 1000 - Loss: 0.006169
Epoch: 2000 - Loss: 0.006147
Epoch: 3000 - Loss: 0.006106
Epoch: 4000 - Loss: 0.006070
Epoch: 5000 - Loss: 0.006065
Epoch: 6000 - Loss: 0.005913
Epoch: 7000 - Loss: 0.005687
Epoch: 8000 - Loss: 0.005861
Epoch: 9000 - Loss: 0.004541
Epoch: 10000 - Loss: 0.003108
Epoch: 11000 - Loss: 0.003656
Epoch: 12000 - Loss: 0.002179
Epoch: 13000 - Loss: 0.001891
Epoch: 14000 - Loss: 0.001503
Epoch: 15000 - Loss: 0.001397
Epoch: 16000 - Loss: 0.001704
Epoch: 17000 - Loss: 0.001140
Epoch: 18000 - Loss: 0.000992
Epoch: 19000 - Loss: 0.003722
Epoch: 20000 - Loss: 0.002510
```

Table of Contents

Conclusions

- Neural networks can be used to learn a solution to PDE.
- There is no need for data for the training.
- PDE is coded into the loss function.
- The method is not yet perfect - some tricks are required to get a good solution.